



**Politecnico  
di Torino**

**Politecnico di Torino**

Corso di Laurea Magistrale

A.a. 2026/2027

# Large Language Models

Docenti:

Flavio Giobergia  
Riccardo Coppola

Studente:

Cristiano Berardo



# Indice

|          |                                                |          |
|----------|------------------------------------------------|----------|
| <b>I</b> | <b>Introduction to LLM</b>                     | <b>1</b> |
| 0.1      | What is a Language Model (LM)?                 | 2        |
| 0.1.1    | Formal definition for language models          | 2        |
| 0.2      | Simple LMs - n-gram models                     | 2        |
| 0.3      | Estimation & generation                        | 2        |
| 0.4      | Generating new sentences                       | 2        |
| 0.5      | Limitation of n-grams                          | 2        |
| 0.6      | Bigger and better Language Models              | 3        |
| <b>1</b> | <b>Introduction to deep learning</b>           | <b>4</b> |
| 1.1      | The perceptron                                 | 4        |
| 1.2      | The perceptron, in 2D                          | 4        |
| 1.3      | Activation functions                           | 4        |
| 1.4      | 1. Enforce properties on perceptron's output   | 5        |
| 1.4.1    | Adding some perceptrons                        | 5        |
| 1.4.2    | and adding other layers!                       | 5        |
| 1.5      | 2. Introduce non-linearities in the model      | 5        |
| 1.6      | Multi-layer perceptron models                  | 5        |
| 1.7      | Activation functions for classification models | 5        |
| 1.8      | Binary classification                          | 6        |
| 1.9      | Multi-class classification                     | 6        |
| 1.10     | Activation functions for regression models     | 6        |
| 1.11     | Defining weights (parameters)                  | 6        |
| 1.12     | Linear regression                              | 6        |
| 1.13     | More complex losses/model                      | 6        |
| 1.13.1   | Limitation of gradient descent                 | 7        |
| 1.14     | Backpropagation                                | 7        |
| 1.14.1   | Using the chain rule                           | 7        |
| 1.14.2   | Computational graph                            | 7        |
| 1.15     | Loss functions                                 | 8        |
| 1.16     | Loss functions                                 | 8        |

# Parte I

## Introduction to LLM

---

## 0.1 What is a Language Model (LM)?

A Language Model is a probabilistic model of a natural language. It produce a probability for any sequence of words.

The probability over all possible sentences in a language sums to 1. There are region of high density in the space of all possible sentences.

### 0.1.1 Formal definition for language models

$$P(w_1, w_2, \dots, w_n) = \prod_{t=1}^T P(w_t | w_1, w_2, \dots, w_{t-1})$$

## 0.2 Simple LMs - n-gram models

We need to estimate the probability of a word given the previous words. A simplifying assumption, Markov assumption, is that:

The next word  $w_t$  depends only on a small windows of  $n-1$  preceding words called the **context**.  
For  $n = 2$ , the next word only depends on the previous word, called a **bigram** model.

$$P(w_t | w_{t-1}, w_{t-2}, \dots, w_1) \approx P(w_t | w_{t-1})$$

## 0.3 Estimation & generation

We can use a language model to estimate probabilities for new sentences. Generate a new sentence based on known probabilities.

- Starting from a given word, we can compute the probability of having any other word following it
- Then, we choose one word based on this probability distribution
- We add the new word to the already available sequence, and generate the next one
- We call this, autoregressive generation

**Example**  $P(\text{the, cat, chased, the, cheese}) = P(\text{the}) \times P(\text{cat}|\text{the}) \times P(\text{chased}|\text{the, cat}) \times P(\text{the}|\text{the, cat, chased}) \times P(\text{cheese}|\text{the, cat, chased, the})$

We can use the Markov assumption to simplify the computation of the probabilities.

$$P(\text{the, cat, chased, the, cheese}) \approx P(\text{the}) \times P(\text{cat}|\text{the}) \times P(\text{chased}|\text{cat}) \times P(\text{the}|\text{chased}) \times P(\text{cheese}|\text{the})$$

Let's assume  $P(\text{the}) = 1$

$$P(\text{the, cat, chased, the, cheese}) \approx 1 \times \frac{1}{4} \times 1 \times 1 \times \frac{1}{4} = \frac{1}{16}$$

## 0.4 Generating new sentences

## 0.5 Limitation of n-grams

1. **Data sparsity:** As  $n$  increases, the number of possible  $n$ -grams grows exponentially. Bigram,  $V$  words, the table size is  $V^2$ .  $n$ -grams,  $V$  words, the table size is  $V^n$ . A small vocabulary or a small context size make the LM quite useless
2. **Context limitation:** Only local context (previous  $n-1$  words) are used to choose. Longer sentences may require remembering what happened "early on"
3. **Lack of semantics:** Similar words are treated in the same way as completely different ones

The language models are not a new concept. In the '90s, researchers tried to build LMs based on existing techniques and computing capabilities. With varying results... Mostly bad.

---

## 0.6 Bigger and better Language Models

We can see  $P(w_t|w_{t-1}, w_{t-2}, \dots, w_1)$  as a classification function.

"Given an **input**, the previous words in the sentence, predict the **next word**, class."

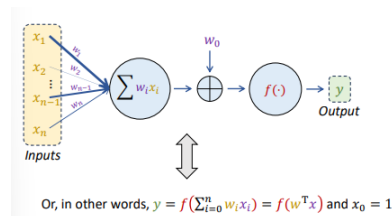
# Capitolo 1

## Introduction to deep learning

### 1.1 The perceptron

The perceptron is the simplest unit of neural networks. It takes an input and **multiplies** it by a weight. Then, it sums all the weighted inputs and applies a non-linear function to produce an output.

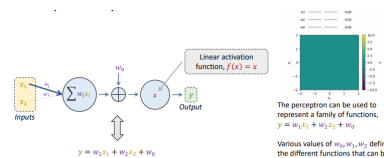
$$y = f(w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n)$$



Where:

- $y$  is the output of the perceptron
- $f$  represents a non-linear “activation” function
- $w_i$  and  $w_0$  are the weights and bias which are learned during training
- $x = (x_1, x_2, \dots, x_n)$  are the inputs to the perceptron

### 1.2 The perceptron, in 2D

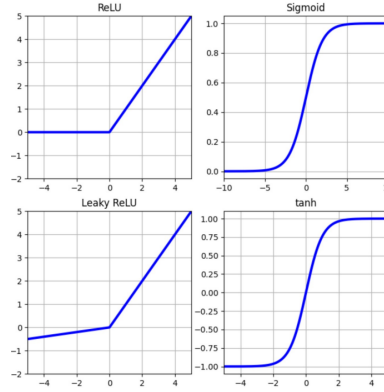


### 1.3 Activation functions

Activation functions are used for two main reasons:

- Enforce properties on perceptron's output. E.g. sigmoid -> binds output to  $[0, 1]$  range
- Introduce non-linearities in the model + some others (faster convergence, sparsity, ...)

Common activation functions are: ReLU, Sigmoid, Leaky ReLU, Tanh, Softmax, Linear, GeLU



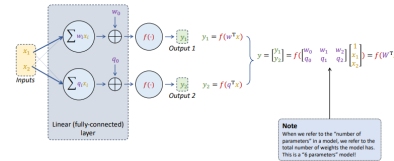
## 1.4 1. Enforce properties on perceptron's output

Binary classification problem: separate positive ( $\star$ ) and negative ( $\circ$ ) samples. For a point  $x \in \mathbb{R}^2$ , the perceptron can predict  $p(\star|x)$ . For a binary case this implies that  $p(\circ|x) = 1 - p(\star|x)$ .

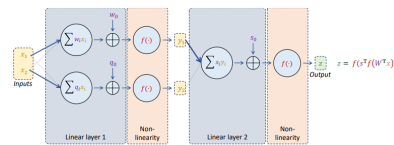
The Sigmoid maps any value in  $\mathbb{R}$  to the range  $[0, 1]$ . The output is squashed between 0 and 1.

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

### 1.4.1 Adding some perceptrons



### 1.4.2 and adding other layers!



## 1.5 2. Introduce non-linearities in the model

This is the second reason why we need activation functions.

If  $f(x) = x$  (i.e., no non-linearity is added) we get  $z = s^T W^T x$

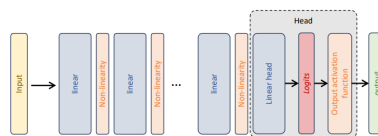
## 1.6 Multi-layer perceptron models

## 1.7 Activation functions for classification models

As argued, activation functions can be used to enforce properties on the model's output

In classification problems, the output before the final activation is treated as unnormalized probabilities **logits unit**.

We still need a step to convert logits into valid probabilities



## 1.8 Binary classification

## 1.9 Multi-class classification

The **softmax** function produce a probability of class (e.g. class 1 (cat), class 2 (dog)), it gives us only values between 0 and 1

$$P(c_i|x) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$



## 1.10 Activation functions for regression models

## 1.11 Defining weights (parameters)

So far, we assumed all weights and biases (let's call them  $\theta$ ) to be known. **But, we still need to figure out how we find them!**

We pick a function, onjective, or loss,  $\mathcal{L}(\theta)$ , that we want to minimize.

It looks at the model, the data are fixed, and give us a number, high number means bad model, low number means good model.

**Example:**

$$\mathcal{L}(\theta) = MSE(\theta) = \frac{1}{n} \sum (y_i - \theta^T x_i)^2$$

Then, we pick  $\theta$  that minimizes it

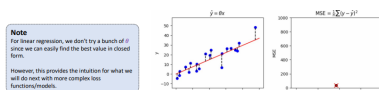
## 1.12 Linear regression

For simple models, we can find the optimal weights in closed form

$$\frac{\partial \mathcal{L}(\theta)}{\partial \theta} = \frac{\partial MSE(\theta)}{\partial \theta} = 0$$

Quadratic in  $\theta$ , can be solved easily!

Or, we can evaluate the loss function for a bunch of  $\theta$ 's, and find the "best" one



## 1.13 More complex losses/model

In more complex loss functions/models, we may not be able to solve the problem in a closed form. But we can evaluate  $\mathcal{L}(\theta)$  for various values of  $\theta$ .

We can iteratively update  $\theta$  to reach a local minimum. We can start from a random value  $\theta$ , then:

- We start from a random value  $\theta$ , then

- we “move around” according to “some policy”

**Move around** = update  $\theta$  incrementally, based on its current value. The new value of  $\theta$  at any step depends on the previous step's value:  $\theta_{t+1} := \theta_t + \text{update}$

**Some policy** = we take a small step in the direction where the function decreases locally. i.e. in the **opposite** direction of the **gradient**.  $\theta_{t+1} := \theta_t - \alpha \nabla_{\theta} \mathcal{L}(\theta_t)$ .

For 1-dimensional  $\theta$ , we have  $\theta_{t+1} := \theta_t - \alpha \frac{\partial \mathcal{L}(\theta)}{\partial \theta}$   $\alpha$ : learning rate, controls the “size” of the step ( $10^{-3}$ ,  $10^{-4}$ , ...)

If the derivative is negative, i move to the right, when the derivative is positive i move to the left and i get the bottom of the function.

Gradient Descent!

### 1.13.1 Limitation of gradient descent

GD is sensitive to weight initialization. Different initializations can lead to different solutions!. GD can get stuck in local minima

Various solutions to help prevent local minima:

- Adding momentum
- Adaptive learning rates, how to regulate the step
- Learning rate schedules

## 1.14 Backpropagation

How to compute the gradient of a loss function with respect to the weights of a neural network?

So far, we assumed we were able to compute  $\nabla_{\theta} \mathcal{L}(\theta)$ .

We need some automated way to compute gradients.

Now we introduce the **backpropagation**, to compute the gradient of the loss w.r.t. any weight! Backpropagation is just a fancy word for “using the chain rule”

### 1.14.1 Using the chain rule

We use the chain rule from calculus

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial g} \cdot \frac{\partial g}{\partial x}$$

Other notation could be  $f(g(x))' = f'(g(x)) \cdot g'(x)$

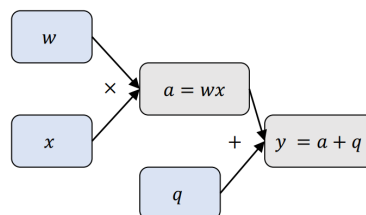
And apply it from the end of the computational graph, backwards. Hence the name, backpropagation.

### 1.14.2 Computational graph

A computational graph is a directed graph

- Each node corresponds to an operation
- Each edge represents the flow of data between nodes

For instance, we may want to compute  $y = wx + b$



We start from three variables,  $w$ ,  $x$  and  $y$ . The computational graph performs one operation at a time.

- First, compute the intermediate variable  $a = wx$
- Then, compute the output variable  $z = a + y = wx + y$

**Backpropagation example** Let's say:

- Our dataset has one point  $(x, y)$
- Our (weird) model has two parameters,  $\theta_1$  and  $\theta_2$ , and predicts  $\theta_1\theta_2x$
- Our loss function is  $L = (\theta_1\theta_2x - y)^2$

We build a computational graph with all operations and intermediate variables.

- $a = \theta_1\theta_2$
- $b = ax = \theta_1\theta_2x$
- $c = b - y = ax - y = \theta_1\theta_2x - y$
- $L = c^2 = (b - y)^2 = (ax - y)^2 = (\theta_1\theta_2x - y)^2$

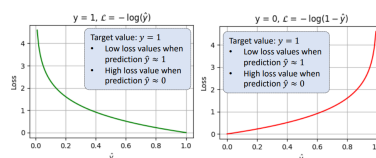
## 1.15 Loss functions

What loss function should I use?

- Regression: Mean Squared Error (MSE), Mean Absolute Error (MAE)
- Binary: Binary Cross Entropy (BCE), the ground truth is  $y = 0, 1$  and the predicted value is  $\hat{y} = \text{model}(x) \in [0, 1]$

$y$ , the ground truth, acts as a "selector" of the loss term to be applied.

- $y = 1 \rightarrow \mathcal{L} = -\log(\hat{y})$
- $y = 0 \rightarrow \mathcal{L} = -\log(1 - \hat{y})$



## 1.16 Loss functions

Multi-class classification

- Cross-Entropy
- Generalization to multiple classes of BCE
- $y_i = 1$  when ground truth is the  $i$ th class, 0 otherwise
- $y_i$  plays the same "selector" mechanism as in BCE

$$\mathcal{L} = - \sum_i y_i \log(\hat{y}_i)$$